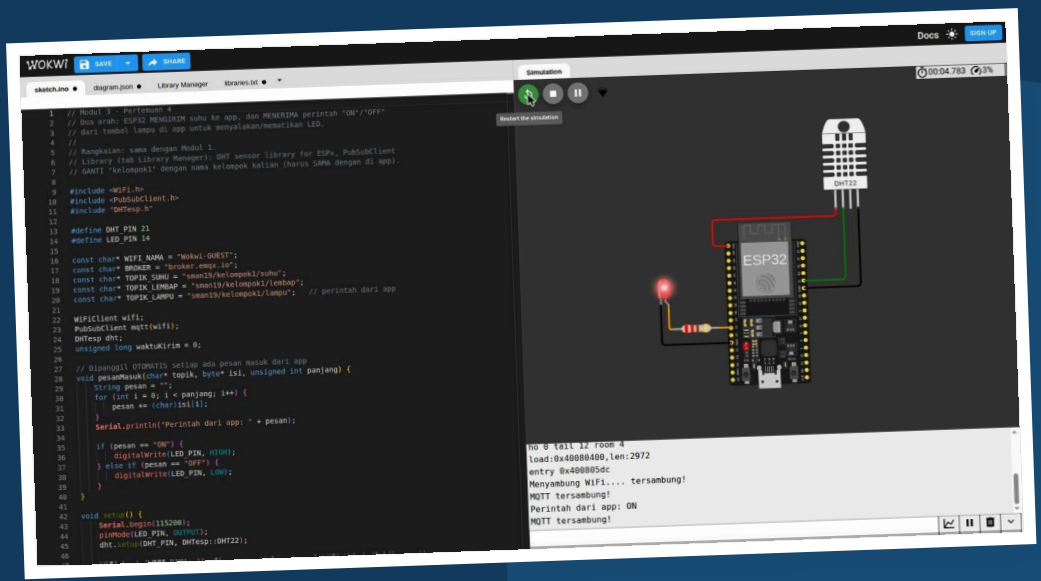
PERTEMUAN 4 · HANDS-ON

INTEGRASI SOFTWARE & HARDWARE

Membuat **"Smart Room"**: ESP32 membaca suhu, mengirimnya ke aplikasi web buatan sendiri, dan lampu bisa dinyalakan dari HP/laptop. Semua lewat simulator, tanpa alat fisik.

Wokwi (ESP32) | Sensor DHT22 | MQTT | Web App HTML



PENGANTAR *& DAFTAR ISI*

Sensor → ESP32 → internet → aplikasi, lalu balik lagi: aplikasi → lampu.

> Yang perlu disiapkan: **laptop + browser Chrome/Edge + internet**. Tidak perlu install apa pun. Semua kode ada di folder `pertemuan4/kode/`, tinggal copy-paste.

Daftar isi

01 Gambaran besar & istilah

02 Persiapan Wokwi

03 Modul 1 — Sensor + LED

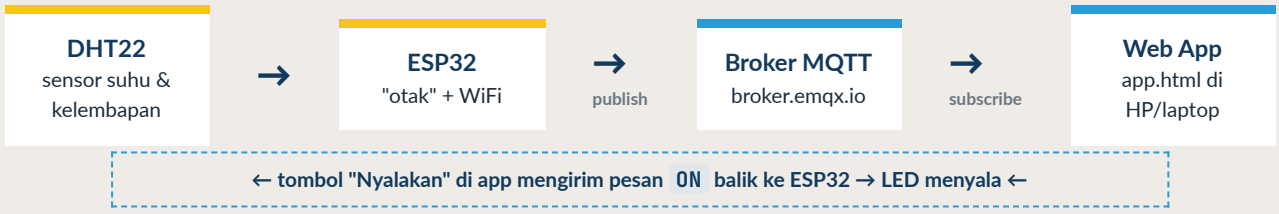
04 Modul 2 — ESP32 → App

05 Modul 3 — App → ESP32

06 Troubleshooting & checklist

1. Gambaran besar

Hari ini kita membuat software dan hardware saling "ngobrol" lewat internet. Itulah inti **IoT (Internet of Things)**.



Istilah	Arti singkat (analogi)
WiFi	Jalan ESP32 menuju internet. Di Wokwi namanya `Wokwi-GUEST`, tanpa password.
MQTT	"Bahasa" kirim pesan pendek antar perangkat IoT. Ringan dan cepat.
Broker	"Kantor pos" yang meneruskan pesan. Kita pakai yang gratis: `broker.emqx.io`.
Topik	"Alamat" pesan, misalnya `sman19/kelompok1/suhu`. Mirip nama grup WA.
Publish	Mengirim pesan ke sebuah topik (ESP32 mengirim suhu).
Subscribe	Berlangganan topik: otomatis menerima setiap pesan baru (app menerima suhu).

Sesi	Kegiatan	Waktu
Pengantar	Gambaran besar + persiapan Wokwi	10 menit
Modul 1	Rangkai sensor DHT22 + LED otomatis	20 menit
Modul 2	ESP32 mengirim suhu ke web app	25 menit
Modul 3	Kontrol lampu dari web app (dua arah)	20 menit
Penutup	Tantangan, troubleshooting, checklist	10 menit

MODUL 1 — *SENSOR + LED*

Pemanasan: pastikan hardware-nya jalan dulu sebelum disambung ke internet.

2. Persiapan Wokwi ± 3 menit

01 Buka **wokwi.com** → pilih **ESP32** → di bagian *Starter Templates* pilih **ESP32** (Arduino). Login tidak wajib.

02 Layar terbagi dua: **kiri** = kode (`sketch.ino`, `diagram.json`, *Library Manager*), **kanan** = simulasi rangkaian + Serial Monitor.

03 Tombol ▶ **hijau** = jalankan simulasi, ■ = stop, tombol **+ biru** = tambah komponen.

3. Rangkai komponen DHT22 + LED + resistor 220Ω

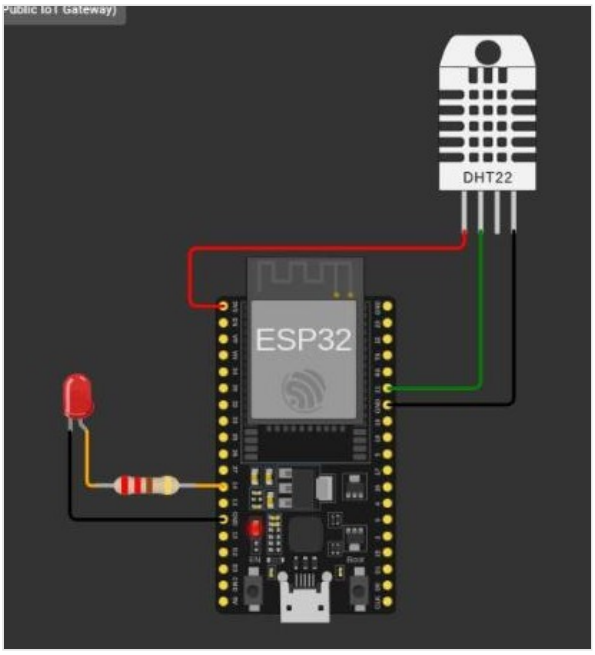
01 Klik **+** → tambahkan **DHT22**, **LED**, dan **Resistor**.

02 Klik resistor → ubah nilainya jadi **220Ω**.

03 Tarik kabel dari kaki komponen ke pin ESP32 sesuai tabel.

04 **Jalan pintas:** buka tab `diagram.json`, ganti seluruh isinya dengan file `kode/diagram.json`.

Kaki komponen	Ke pin ESP32
DHT22 VCC	3V3
DHT22 SDA	21
DHT22 GND	GND
LED A (+, kaki panjang)	resistor → 14
LED C (−, kaki pendek)	GND



Gambar 1. Rangkaian jadi di Wokwi. Kabel merah = daya (3V3), hitam = GND, hijau = data sensor, oranye = sinyal LED.

> **Ingat Pertemuan 3:** LED itu berpolaritas. Kaki **A** (+) ke pin lewat resistor, kaki **C** (−) ke GND. Kalau terbalik, LED tidak akan menyala.

4. Tambah library sensor

01 Klik tab **Library Manager** → tombol **+** → ketik `DHT sensor library for ESPx` → klik hasilnya.

5. Kode Modul 1 kode/modul1_sensor.ino

```ino
#include "DHTesp.h"

#define DHT_PIN 21
#define LED_PIN 14

DHTesp dht;

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  dht.setup(DHT_PIN, DHTesp::DHT22);
}

void loop() {
  float suhu = dht.getTemperature();
  float lembap = dht.getHumidity();

  Serial.print("Suhu: ");
  Serial.print(suhu);
  Serial.print(" C | Kelembapan: ");
  Serial.print(lembap);
  Serial.println(" %");

  if (suhu > 30) {
    digitalWrite(LED_PIN, HIGH); // panas
  } else {
    digitalWrite(LED_PIN, LOW);
  }
  delay(2000);   // baca tiap 2 detik
}
```

Apa yang baru?

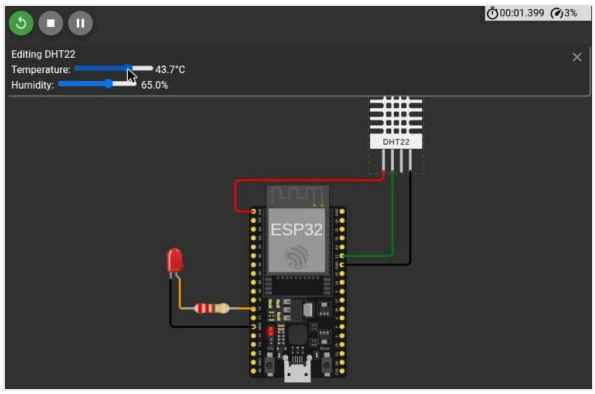
- `dht.setup(pin, tipe)` : memberi tahu ESP32 sensor ada di pin mana.
- `dht.getTemperature()` : membaca suhu (°C), hasilnya `float`.
- `if (suhu > 30)` : `if-else` dari Pertemuan 1, sekarang memutuskan berdasarkan data sensor sungguhan.

Langkah uji coba

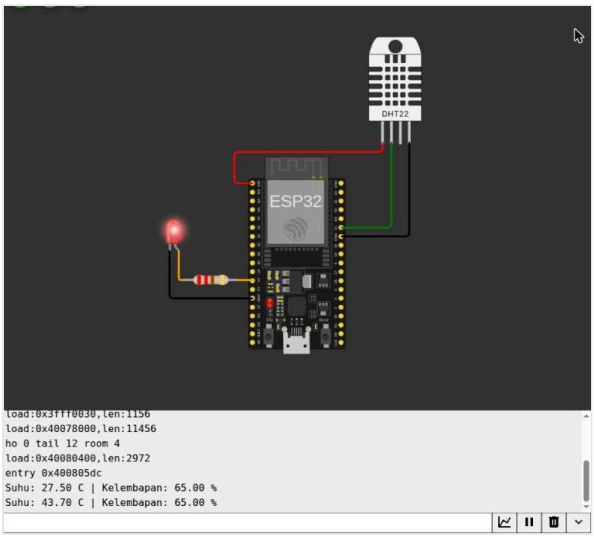
01 Paste kode ke `sketch.ino`, klik ▶.

02 Serial Monitor menampilkan suhu tiap 2 detik.

03 **Bagian seru:** klik sensor DHT22 lalu geser slider **Temperature** ke atas 30°C. LED langsung menyala!



Gambar 2. Klik DHT22 untuk memunculkan slider suhu & kelembapan.



Gambar 3. Suhu 43,7°C > 30°C → LED menyala. Serial Monitor mencatat perubahannya.

★ TANTANGAN MODUL 1

Tambahkan kondisi kedua: kalau **kelembapan > 80%**, LED **kedip-kedip** (nyala 200 ms, mati 200 ms) sebagai tanda "ruangan lembap". Petunjuk: pakai `else if` dan `delay(200)`.

MODUL 2 — *ESP32 → APP*

ESP32 tersambung WiFi lalu "mem-publish" suhu ke broker. Web app "subscribe" dan menampilkannya.

6. Siapkan ESP32

01 Library Manager → + → tambahkan juga `PubSubClient` (library MQTT).

02 Ganti isi `sketch.ino` dengan `kode/modul2_kirim_ke_app.ino`.

03 Ganti `kelompok1` di baris `TOPIK_...` dengan **nama kelompok kalian**, misalnya `kel3-ayu-budi`.

> **Kenapa nama kelompok harus unik?** Broker gratis ini dipakai orang di seluruh dunia. Kalau semua memakai `kelompok1`, data kalian akan tercampur dengan kelompok lain.

BAGIAN PENTING KODE MODUL 2

```
#include <WiFi.h>
#include <PubSubClient.h>
#include "DHTesp.h"

const char* WIFI_NAMA  = "Wokwi-GUEST";       // WiFi virtual milik Wokwi
const char* BROKER     = "broker.emqx.io";    // "kantor pos" MQTT gratis
const char* TOPIK_SUHU = "sman19/kelompok1/suhu";

void setup() {
  ...
  WiFi.begin(WIFI_NAMA, "", 6);            // 1. sambung WiFi (channel 6 = lebih cepat)
  while (WiFi.status() ≠ WL_CONNECTED) {   //    tunggu sampai tersambung
    delay(300); Serial.print(".");
  }
  mqtt.setServer(BROKER, 1883);             // 2. alamat kantor pos
}

void loop() {
  if (!mqtt.connected()) {                   // 3. masuk ke broker (sekali saja)
    mqtt.connect(id.c_str());
  }
  float suhu = dht.getTemperature();
  mqtt.publish(TOPIK_SUHU, String(suhu, 1).c_str());   // 4. KIRIM! (1 angka desimal)
  delay(2000);
}
```



Gambar 4. Serial Monitor ketika berhasil: `Menyambung WiFi.... tersambung!` → `MQTT tersambung!` → `Terkirim → suhu ...`. Ikon WiFi di samping tombol simulasi juga menunjukkan status koneksi.

7. Jalankan web app "Smart Room"

01 Klik ▶ di Wokwi dan tunggu sampai `MQTT tersambung!`.

02 Buka file `kode/app.html` (klik 2×, terbuka di Chrome/Edge).

03 Isi **Nama kelompok** persis sama dengan di kode ESP32, lalu klik **Sambungkan**.

04 Titik status berubah hijau, lalu angka suhu dan kelembapan muncul dan diperbarui tiap 2 detik.

05 **Bagian seru:** geser slider DHT22 di Wokwi, dan angka di app ikut berubah. Kalau suhu > 30°C, app menampilkan peringatan kuning.

> **Mau dibuka di HP?** Kirim file `app.html` ke HP lalu buka dengan Chrome. Karena lewat internet, HP **tidak perlu** satu WiFi dengan laptop.



Gambar 5. Web app menerima data asli dari ESP32 di Wokwi: suhu 43,7°C, kelembapan 65%. Log pesan mencatat jam setiap pesan masuk.

8. Intip isi app.html bagian yang "mendengarkan" ESP32

```
<!-- library MQTT untuk browser -->
<script src="https://unpkg.com/mqtt@5.10.1/dist/mqtt.min.js"></script>

const BROKER = "wss://broker.emqx.io:8084/mqtt";   // broker yang SAMA, lewat WebSocket
klien = mqtt.connect(BROKER);
klien.subscribe(topik + "/#");                     // langganan semua topik kelompok

klien.on("message", (t, isi) => {                    // dipanggil tiap ada pesan masuk
  if (t === topik + "/suhu") {
    document.getElementById("suhu").textContent = isi;   // tampilkan di layar
  }
});
```

ESP32 memakai port **1883** (MQTT biasa), sedangkan browser memakai **8084** (MQTT lewat WebSocket aman). Brokernya sama, jadi pesan tetap bertemu.

★ TANTANGAN MODUL 2

Di ESP32, kirim satu data lagi: teks `"PANAS"` atau `"NORMAL"` ke topik `sman19/<kelompok>/status` (pakai `if-else`). Lalu cek: apakah pesan itu muncul di **Log pesan** app? Kenapa bisa muncul padahal app tidak diubah? (Petunjuk: lihat `"/#"`.)

MODUL 3 — *APP → ESP32*

Sekarang arah sebaliknya: tombol di app mengirim perintah, ESP32 menerimanya, LED menyala.

9. Langkah

01 Ganti isi `sketch.ino` dengan `kode/modul3_kontrol_dari_app.ino`. Ganti `kelompok1` di **3 baris** topik.

02 Klik ▶ dan tunggu `MQTT tersambung!`. App yang sudah terbuka tidak perlu di-refresh.

03 Klik **Nyalakan** di app: LED di Wokwi menyala dan Serial Monitor menulis `Perintah dari app: ON`. Klik **Matikan** untuk mematikannya.

TAMBAHAN KODE MODUL 3

```
const char* TOPIK_LAMPU = "sman19/kelompok1/lampu";   // perintah dari app

// Dipanggil OTOMATIS setiap ada pesan masuk dari app
void pesanMasuk(char* topik, byte* isi, unsigned int panjang) {
  String pesan = "";
  for (int i = 0; i < panjang; i++) {      // rangkai huruf demi huruf
    pesan += (char)isi[i];
  }
  if (pesan == "ON") {
    digitalWrite(LED_PIN, HIGH);
  } else if (pesan == "OFF") {
    digitalWrite(LED_PIN, LOW);
  }
}

// di setup():
mqtt.setCallback(pesanMasuk);       // daftarkan fungsi penerima pesan
// di loop(), setelah connect:
mqtt.subscribe(TOPIK_LAMPU);        // "langganan" topik lampu
mqtt.loop();                         // cek pesan masuk, jangan diblok delay!

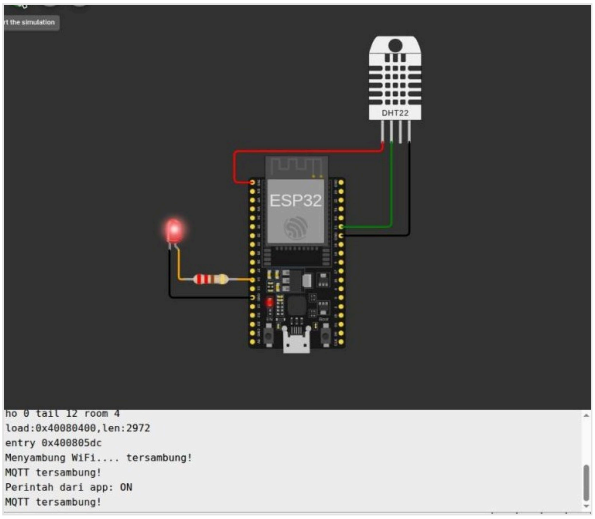
if (millis() - waktuKirim > 2000) {   // kirim suhu tiap 2 detik TANPA delay()
  waktuKirim = millis();
  ...publish suhu & lembap...
}
```

Konsep baru	Penjelasan
Callback	Fungsi yang **tidak kita panggil sendiri**. Library yang memanggilnya otomatis saat ada pesan. Isinya tetap `for` + `if-else` dari Pertemuan 1.
`millis()`	"Stopwatch" ESP32: jumlah milidetik sejak menyala. Dipakai mengganti `delay(2000)` supaya `mqtt.loop()` terus jalan dan tombol app langsung direspons.

10. Hasil: lampu dikontrol dari aplikasi



Gambar 6. Tombol **Nyalakan** ditekan: bohlam di app menyala kuning, status **NYALA**, dan log mencatat `lampu = ON`.



Gambar 7. Di saat yang sama, LED di Wokwi menyala dan Serial Monitor menulis `Perintah dari app: ON`.

Coba bareng teman: buka app di HP teman dengan nama kelompok yang sama. Kalau teman menekan tombol, LED di laptop kalian ikut menyala. Itulah kenapa nama kelompok harus dijaga "rahasia" 😉

★ TANTANGAN MODUL 3

Tambahkan tombol **"Kedip"** di `app.html`:
`<button onclick="kirimLampu('BLINK')">Kedip</button>`
Lalu di fungsi `pesanMasuk` tambahkan `else if (pesan == "BLINK")` yang membuat LED kedip **5 kali** memakai `for` loop (seperti studi kasus LED Pertemuan 2).

★★ TANTANGAN BONUS: MODE OTOMATIS

Gabungkan Modul 1 dan 3: tambahkan perintah `"AUTO"`. Saat mode AUTO aktif, LED dikendalikan suhu (menyala kalau > 30°C). Saat app mengirim `ON` / `OFF`, mode kembali manual. Petunjuk: pakai variabel `bool modeAuto = false;`.

TROUBLESHOOTING & CHECKLIST

Kalau ada yang tidak jalan, cek tabel ini dulu sebelum bertanya.

11. Masalah yang sering terjadi

Gejala	Solusi
Lama di "Waiting in queue"	Server build Wokwi sedang antre (satu kelas menekan ▶ bersamaan). Tunggu sebentar, atau login Wokwi (gratis 1× lewati antrean per hari).
`Menyambung WiFi.....` **tidak selesai**	Pastikan nama WiFi persis `Wokwi-GUEST` dan password `""`. Arahkan mouse ke ikon WiFi di samping tombol simulasi untuk melihat pesan error, lalu klik restart.
Error `PubSubClient.h: No such file`	Library belum ditambahkan di Library Manager.
App "Tersambung" tapi angka tetap `--`	Nama kelompok di app berbeda dengan di kode ESP32. Huruf besar/kecil dan spasi berpengaruh!
Tombol app tidak berpengaruh	Pastikan memakai kode **Modul 3** (Modul 2 belum `subscribe`) dan topik lampu sudah diganti.
Suhu terbaca `nan`	Kabel SDA DHT22 tidak ke pin 21, atau nilai `DHT_PIN` salah.
Simulasi terasa lambat	Normal untuk simulasi WiFi. Tutup tab lain, jangan pindah tab saat simulasi berjalan.

12. Rangkuman

Hardware (sensor, LED) dibaca dan dikendalikan oleh **ESP32**. ESP32 tersambung **WiFi**, lalu **publish** data ke **broker MQTT**. Web app **subscribe** topik yang sama sehingga menerima data, dan bisa **publish** perintah balik yang diterima ESP32 lewat fungsi **callback**. Logikanya tetap `setup()` / `loop()`, `if-else`, dan `for` yang sama seperti Pertemuan 1–3. Yang baru hanya "jalurnya" sekarang lewat internet.

13. Checklist self-check

- ☐ Saya bisa merangkai DHT22 + LED ke ESP32 di Wokwi.
- ☐ Saya bisa menjelaskan arti broker, topik, publish, dan subscribe dengan bahasa sendiri.
- ☐ Suhu dari Wokwi muncul di web app saya, dan ikut berubah saat slider digeser.
- ☐ Saya bisa menyalakan dan mematikan LED Wokwi dari tombol di web app.
- ☐ Saya tahu kenapa Modul 3 memakai `millis()`, bukan `delay()`.
- ☐ Saya sudah mencoba minimal 1 soal tantangan.

> Semua file ada di `pertemuan4/kode/`: `diagram.json`, `modul1_sensor.ino`, `modul2_kirim_ke_app.ino`, `modul3_kontrol_dari_app.ino`, `app.html`.